

Hochschule Mainz
Fachbereich Technik
Studiengang [Ihr Studiengang]

Überladen von Operatoren in C++

Konzepte, Implementierung und Effizienzanalyse

Seminararbeit
im Modul **Effiziente Programmierung (C/C++)**
Thema 16

Vorgelegt von: [Vor- und Nachname]
Matrikelnummer: [Matrikelnummer]
E-Mail: [E-Mail-Adresse]

Betreuung: Jessica Buchner

Abgabedatum: 14.06.2026

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Problemstellung	1
1.2	Zielsetzung	1
1.3	Aufbau der Arbeit	1
2	Theoretischer Hintergrund	2
2.1	Grundlagen der Operatorüberladung	2
2.2	Member- vs. Nicht-Member-Funktionen	3
2.3	Spezielle Operatoren	4
2.4	Rule of Three / Five / Zero	4
2.5	Kopier- und Move-Semantik	5
2.6	Return Value Optimization (RVO/NRVO)	6
2.7	Operatorüberladung in anderen Sprachen	7
3	Praktische Implementierung	7
3.1	Beschreibung der Beispielklasse	8
3.2	Version 1: Naive Implementierung	8
3.3	Version 2: Implementierung mit Move-Semantik	9
3.4	Version 3: +=-basierte Implementierung	10
3.5	Instrumentierung zur Zählung temporärer Objekte	10
4	Experimente und Analyse	11
4.1	Versuchsaufbau	11
4.2	Messmethodik und Reproduzierbarkeit	11
4.3	Ergebnisse: Laufzeit	12
4.4	Ergebnisse: Temporäre Objekte	14
4.5	Ergebnisse: Speicherverbrauch	15
4.6	Interpretation der Ergebnisse	16
5	Fazit	17
5.1	Zusammenfassung	17
5.2	Empfehlungen	18
5.3	Ausblick	18

1 Einleitung

1.1 Motivation und Problemstellung

Das Überladen von Operatoren gehört zu den charakteristischen Sprachmerkmalen von C++. Es erlaubt es, die in der Sprache vordefinierten Operatoren wie `+`, `-`, `*`, `==` oder `[]` auch auf selbst definierte Datentypen anzuwenden. Statt eine Addition zweier Matrizen über einen Methodenaufruf wie `A.add(B)` auszudrücken, kann eine benutzerdefinierte Klasse so gestaltet werden, dass der intuitive Ausdruck `A + B` möglich wird. Auf diese Weise verhalten sich eigene Typen syntaktisch wie eingebaute Typen, was die Lesbarkeit und Ausdrucksstärke des Quellcodes erhöht [1, 5].

Diese syntaktische Eleganz ist jedoch nicht kostenlos. Hinter einem schlicht aussehenden Ausdruck wie `D = A + B + C` können sich zahlreiche implizite Operationen verbergen: die Erzeugung temporärer Objekte, das Kopieren großer Datenmengen sowie wiederholte Speicheranforderungen auf dem Heap. Gerade weil diese Kosten im Quelltext nicht unmittelbar sichtbar sind, besteht die Gefahr, dass ineffizienter Code entsteht, ohne dass dies dem Entwickler bewusst wird. Im Kontext der effizienten Programmierung ist die Operatorüberladung somit ein anschauliches Beispiel dafür, dass lesbarer Code und performanter Code nicht zwangsläufig identisch sind, sich durch geeignete Implementierungstechniken aber miteinander vereinbaren lassen.

Zentral für diese Vereinbarkeit sind moderne C++-Konzepte wie die Move-Semantik, rvalue-Referenzen sowie compilerseitige Optimierungen wie die Return Value Optimization (RVO). Sie ermöglichen es, die durch Operatorüberladung entstehenden temporären Objekte zu vermeiden oder kostengünstig zu verschieben, anstatt sie aufwändig zu kopieren. Welchen konkreten Einfluss unterschiedliche Implementierungsvarianten auf Laufzeit, Speicherverbrauch und die Anzahl temporärer Objekte haben, bildet den Kern der vorliegenden Untersuchung.

Für die Einordnung der Messergebnisse ist dabei ein Standarddetail zentral: Seit C++17 wird bei der Initialisierung eines neuen Objekts aus einem prvalue – etwa in `Matrix D = A + B;` – der Rückgabewert verpflichtend ohne zusätzliche Kopie oder Move materialisiert ("guaranteed copy elision"). Dadurch unterscheiden sich naive und move-fähige Implementierungen in genau diesem Fall oft kaum, selbst bei -O0. Relevante Unterschiede durch Move-Semantik zeigen sich vor allem bei der Zuweisung aus Temporaries (z. B. `C = A + B;`), bei verketteten Ausdrücken mit echten Zwischenwerten (z. B. `A + B + C`) und bei Container-Reallokationen [6, 3].

1.2 Zielsetzung

Ziel dieser Arbeit ist es, das Überladen von Operatoren in C++ sowohl theoretisch fundiert darzustellen als auch dessen Effizienzaspekte empirisch zu untersuchen. Anhand einer beispielhaften Klasse werden mehrere Implementierungsvarianten desselben Operators entwickelt und systematisch miteinander verglichen. Im Mittelpunkt stehen dabei die folgenden Fragen: Wie wirken sich die naive Implementierung, die Nutzung der Move-Semantik sowie eine auf zusammengesetzten Zuweisungsoperatoren basierende Variante auf die Performance aus? Welche Rolle spielen dabei die Optimierungsstufen des Compilers? Und in welchem Umfang lassen sich temporäre Objekte durch geeignete Techniken tatsächlich einsparen? Die Arbeit verfolgt damit das Ziel, ein fundiertes Verständnis für die Effizienzimplikationen der Operatorüberladung zu vermitteln und daraus praktische Empfehlungen abzuleiten.

1.3 Aufbau der Arbeit

Die Arbeit gliedert sich in vier Hauptteile. Kapitel 2 legt die theoretischen Grundlagen der Operatorüberladung dar und behandelt unter anderem die Unterscheidung zwischen Member- und Nicht-Member-Funktionen, die Rule of Three/Five/Zero sowie die Move-Semantik. Kapitel 3 beschreibt die praktische

Umsetzung der verschiedenen Implementierungsvarianten einschließlich einer Instrumentierung zur Zählung temporärer Objekte. Kapitel 4 stellt den Versuchsaufbau vor und präsentiert sowie interpretiert die Messergebnisse hinsichtlich Laufzeit, Speicherverbrauch und temporärer Objekte. Kapitel 5 fasst die Erkenntnisse zusammen und gibt Empfehlungen für den praktischen Einsatz der Operatorüberladung.

2 Theoretischer Hintergrund

2.1 Grundlagen der Operatorüberladung

In C++ ist ein Operator im Kern nichts anderes als eine Funktion mit einer besonderen, vom Compiler erkannten Schreibweise. Wenn der Compiler einen Ausdruck wie `a + b` verarbeitet, übersetzt er ihn intern in einen Funktionsaufruf – entweder `a.operator+(b)`, falls der Operator als Member-Funktion definiert ist, oder `operator+(a, b)`, falls er als freie Funktion vorliegt. Das Überladen von Operatoren bedeutet daher lediglich, für einen oder beide Operanden eines benutzerdefinierten Typs eine solche `operator`-Funktion bereitzustellen. Die Syntax folgt dem Schema `Rückgabotyp operator<symbol>(Parameter)`, wobei `<symbol>` für den jeweiligen Operator steht, etwa `+`, `==` oder `[]`.

Diese Grundlagen und die zugehörigen Sprachregeln sind in der Standardliteratur sowie in der Referenzdokumentation umfassend beschrieben [1, 5].

Wichtig ist, dass mindestens ein Operand ein benutzerdefinierter Typ (also eine Klasse, Struktur oder ein Enum) sein muss. Es ist nicht möglich, das Verhalten von Operatoren für ausschließlich eingebaute Typen zu verändern – ein Ausdruck wie `1 + 2` lässt sich nicht umdefinieren. Diese Einschränkung verhindert, dass die fundamentale Semantik der Sprache durch eigene Definitionen unterlaufen wird.

Nicht jeder Operator kann überladen werden. Die überwiegende Mehrheit ist überladbar, darunter die arithmetischen Operatoren (`+`, `-`, `*`, `/`, `%`), die Vergleichsoperatoren (`==`, `!=`, `<`, `>`, `<=`, `>=`), die logischen Operatoren, der Indexzugriff `[]`, der Funktionsaufrufoperator `()`, der Dereferenzierungsoperator `*` und der Pfeiloperator `->`, die Zuweisungsoperatoren sowie die Stream-Operatoren `<<` und `>>`. Einige wenige Operatoren sind hingegen ausdrücklich von der Überladung ausgeschlossen: der Bereichsauflösungsoperator `::`, der Member-Zugriff über Punkt `.`, der Member-Pointer-Zugriff `.*`, der ternäre Bedingungsoperator `?:` sowie `sizeof` und `typeid`. Der Grund liegt darin, dass diese Operatoren auf einer so grundlegenden Ebene der Sprache arbeiten, dass eine Umdefinition zu nicht auflösbaren Mehrdeutigkeiten oder zum Verlust der Typsicherheit führen würde.

Bei der Überladung gibt es eine zentrale Regel, die zwar nicht vom Compiler erzwungen, in der Praxis aber als verbindlich gilt: Ein überladener Operator sollte sich semantisch so verhalten, wie es von ihm intuitiv erwartet wird. Ein `operator+` sollte etwas addieren oder zusammenfügen und keine Seiteneffekte auslösen, die den Zustand der Operanden verändern. Wird diese Konvention verletzt – etwa indem `operator+` in Wahrheit eine Subtraktion durchführt – bleibt der Code zwar lauffähig, wird aber unleserlich und fehleranfällig. Die Stärke der Operatorüberladung liegt gerade darin, dass eigene Typen sich erwartungskonform verhalten; diese Erwartungskonformität ist Voraussetzung für ihren sinnvollen Einsatz.

Ein weiterer grundlegender Aspekt betrifft die Frage, was ein Operator zurückgeben sollte. Hier hat sich eine Konvention etabliert, die eng mit der Effizienz zusammenhängt und die in Kapitel 4 empirisch untersucht wird: Operatoren, die ein neues Objekt erzeugen (wie `operator+`), geben üblicherweise per Wert zurück, da das Ergebnis ein eigenständiges, neues Objekt ist. Operatoren hingegen, die einen bestehenden Operanden verändern (wie `operator+=`), geben üblicherweise eine Referenz auf diesen Operanden zurück, um Verkettungen wie `a += b += c` zu ermöglichen und unnötige Kopien zu vermeiden. Diese Unterscheidung zwischen Wert- und Referenzrückgabe ist nicht nur eine Stilfrage, sondern hat unmittelbare Auswirkungen auf die Anzahl der erzeugten temporären Objekte und damit auf die Performance.

2.2 Member- vs. Nicht-Member-Funktionen

Ein überladener Operator kann auf zwei Arten implementiert werden: als Member-Funktion der Klasse oder als freie (Nicht-Member-)Funktion außerhalb der Klasse. Beide Varianten sind funktional weitgehend gleichwertig, unterscheiden sich jedoch in wichtigen Details, die die Wahl im Einzelfall bestimmen.

Wird ein Operator als Member-Funktion definiert, so ist der linke Operand stets das Objekt, auf dem die Funktion aufgerufen wird (`*this`), während der rechte Operand als Parameter übergeben wird. Ein binärer Operator wie `+` benötigt als Member daher nur einen einzigen Parameter. Der Ausdruck `a + b` wird in diesem Fall zu `a.operator+(b)` übersetzt. Der entscheidende Vorteil der Member-Variante ist der direkte Zugriff auf die privaten Datenmember der Klasse, ohne dass eine zusätzliche `friend`-Deklaration nötig wäre.

Wird der Operator hingegen als freie Funktion definiert, so werden beide Operanden als explizite Parameter übergeben. Ein binärer Operator benötigt in dieser Form zwei Parameter, und der Ausdruck `a + b` wird zu `operator+(a, b)`. Da eine freie Funktion außerhalb der Klasse steht, hat sie standardmäßig keinen Zugriff auf deren private Member. Benötigt sie diesen Zugriff, muss sie innerhalb der Klasse als `friend` deklariert werden, oder die Klasse muss öffentliche Zugriffsmethoden bereitstellen.

Die Wahl zwischen beiden Formen folgt einigen etablierten Faustregeln. Operatoren, die den linken Operanden zwingend verändern und untrennbar mit dem Objekt verbunden sind, werden in der Regel als Member implementiert. Dazu gehören insbesondere die zusammengesetzten Zuweisungsoperatoren wie `+=` und `-=`. Für den Zuweisungsoperator `=`, den Indexoperator `[]`, den Funktionsaufrufoperator `()` sowie den Pfeiloperator `->` ist die Member-Form sogar zwingend vorgeschrieben.

Symmetrische binäre Operatoren wie `+`, `-`, `*` und insbesondere die Vergleichsoperatoren werden hingegen häufig als freie Funktionen bevorzugt. Der Grund dafür liegt in der Behandlung impliziter Typumwandlungen. Ist `operator+` als Member definiert, so wird der linke Operand bevorzugt behandelt: Eine implizite Umwandlung kann nur auf den als Parameter übergebenen rechten Operanden, nicht aber auf das aufrufende Objekt angewendet werden. Konkret bedeutet das: Existiert für eine Klasse `Number` ein Konstruktor, der aus einem `int` ein `Number`-Objekt erzeugt, so funktioniert bei der Member-Variante zwar der Ausdruck `n + 5` (der `int` wird in ein `Number` umgewandelt), nicht aber `5 + n`, da auf das linke Literal `5` keine Member-Funktion aufgerufen werden kann. Wird `operator+` hingegen als freie Funktion definiert, sind beide Operanden gleichberechtigt, und beide Ausdrücke `n + 5` ebenso wie `5 + n` lassen sich durch implizite Umwandlung auflösen. Diese Symmetrie ist der Hauptgrund, weshalb symmetrische arithmetische Operatoren und Vergleichsoperatoren konventionell als freie Funktionen implementiert werden.

Ein weiterer, häufig auftretender Fall sind die Stream-Operatoren `<<` und `>>`. Diese müssen zwingend als freie Funktionen implementiert werden, da ihr linker Operand ein Stream-Objekt (etwa `std::ostream`) ist und nicht der eigene benutzerdefinierte Typ. Eine Member-Implementierung würde voraussetzen, dass man die Klasse `std::ostream` selbst um eine Methode erweitern könnte, was nicht möglich ist. Da der Operator für eine korrekte Ausgabe in der Regel auf die internen Daten des eigenen Typs zugreifen muss, wird er üblicherweise als `friend` deklariert.

Eine in modernem C++ verbreitete Entwurfstechnik kombiniert beide Welten und ist zugleich für die Effizienzbetrachtung dieser Arbeit relevant: Der zusammengesetzte Operator `+=` wird als Member implementiert, da er den linken Operanden ohnehin verändert. Der symmetrische Operator `+` wird anschließend als freie Funktion definiert, die intern auf `+=` zurückgreift. Auf diese Weise wird die eigentliche Logik nur an einer Stelle gepflegt, Codeduplizierung vermieden und gleichzeitig die für symmetrische Operatoren wünschenswerte Behandlung beider Operanden gewährleistet. Diese Variante entspricht der in Kapitel 3 vorgestellten Version 3 und wird dort hinsichtlich ihrer Performance mit den übrigen Implementierungen verglichen.

Zusammenfassend lässt sich festhalten, dass die Entscheidung zwischen Member- und freier Funktion weniger eine Frage des persönlichen Stils als vielmehr eine Konsequenz aus der Semantik des jeweiligen

Operators ist. Operatoren, die den linken Operanden verändern oder fest an ihn gebunden sind, gehören als Member in die Klasse; symmetrische Operatoren, die beide Operanden gleichberechtigt behandeln sollen, sind als freie Funktionen besser aufgehoben.

Diese Entwurfsprinzipien entsprechen etablierten Empfehlungen zur Operatorgestaltung in modernem C++ [2, 8, 5].

2.3 Spezielle Operatoren

Einige Operatoren verdienen besondere Aufmerksamkeit, weil ihre Verwendung häufig mit API-Design-Entscheidungen verknüpft ist. Der Zuweisungsoperator `operator=` sollte Selbstzuweisung korrekt behandeln und eine Referenz auf `*this` zurückgeben, damit Verkettungen wie `a = b = c;` funktionieren. Der Indexoperator `operator[]` wird in der Praxis meist in zwei Varianten angeboten, als konstante und als nicht-konstante Überladung, damit sowohl lesender als auch schreibender Zugriff effizient möglich ist.

Bei den Vergleichsoperatoren hat C++20 die Implementierung deutlich vereinfacht: Mit dem Drei-Wege-Vergleich `operator<=>` kann die totale Ordnung zentral beschrieben werden; daraus können klassische Operatoren wie `<`, `<=`, `>` und `>=` automatisch abgeleitet werden. Zusätzlich lassen sich `operator==` und `operator<=>` häufig mit `= default` deklarieren, wenn alle relevanten Member vergleichbar sind. Das reduziert Boilerplate und verbessert die Konsistenz des Vergleichsverhaltens.

Ein klassischer Sonderfall betrifft die logischen Operatoren `&&` und `||`: Werden sie überladen, geht die eingebaute Kurzschlussauswertung verloren. Beide Operanden werden dann ausgewertet, was leicht zu unerwarteten Seiteneffekten und Performance-Nachteilen führt. Deshalb werden diese Operatoren für benutzerdefinierte Typen nur in Ausnahmefällen überladen.

2.4 Rule of Three / Five / Zero

Jede C++-Klasse verfügt über eine Reihe spezieller Elementfunktionen, die der Compiler automatisch erzeugen kann: den Destruktor, den Kopierkonstruktor, den Kopierzweisungsoperator sowie – seit C++11 – den Move-Konstruktor und den Move-Zuweisungsoperator. Die sogenannten “Rules” sind Leitlinien dafür, wann ein Programmierer diese Funktionen selbst definieren muss, anstatt sich auf die vom Compiler erzeugten Varianten zu verlassen.

Rule of Three. Die Rule of Three besagt, dass eine Klasse, die einen benutzerdefinierten Destruktor, Kopierkonstruktor oder Kopierzweisungsoperator benötigt, nahezu immer alle drei benötigt. Der Grund liegt in der Ressourcenverwaltung. Eine Klasse, die eine rohe Ressource verwaltet – typischerweise einen mit `new[]` angeforderten Speicherblock auf dem Heap –, muss diesen in ihrem Destruktor wieder freigeben. Die vom Compiler erzeugten Kopieroperationen führen jedoch eine flache Kopie durch: Sie duplizieren den Zeiger, nicht den Speicherbereich, auf den er verweist. Zwei Objekte verweisen dann auf denselben Block, was zu zwei unabhängigen Destruktoraufrufen führt, die denselben Speicher freigeben (*double free*), sowie zu Aliasing, bei dem die Änderung eines Objekts unbemerkt das andere verändert. Korrektes Verhalten erfordert eine tiefe Kopie: das Anlegen eines neuen Speicherblocks und das Kopieren der Elemente. Die Rule of Three koppelt die drei Funktionen daher aneinander – wer eine benötigt, benötigt alle [2, 8].

Rule of Five. Mit C++11 wurde die Move-Semantik eingeführt und mit ihr zwei weitere spezielle Elementfunktionen: der Move-Konstruktor und der Move-Zuweisungsoperator. Die Rule of Five erweitert die Rule of Three um diese beiden: Eine ressourcenverwaltende Klasse sollte alle fünf Funktionen deklarieren. Über die bloße Symmetrie hinaus steht dahinter ein subtiler, aber folgenreicher Grund. Der Compiler erzeugt für eine Klasse, die einen Destruktor (oder eine Kopieroperation) deklariert, keine Move-Operationen. Eine Klasse, die nur der Rule of Three folgt – also einen Destruktor deklariert –, verliert somit stillschweigend ihre Move-Operationen und fällt überall dort auf das Kopieren zurück,

wo ein Verschieben möglich gewesen wäre. Um die Effizienz der Move-Semantik zurückzugewinnen, müssen Move-Konstruktor und Move-Zuweisungsoperator explizit deklariert werden. Beide sollten als `noexcept` gekennzeichnet sein: Standardcontainer wie `std::vector` verschieben ihre Elemente bei einer Reallokation nur dann, wenn der Move-Konstruktor `noexcept` ist; andernfalls kopieren sie, um die starke Ausnahmegarantie zu wahren. Ein fehlendes `noexcept` führt somit unbemerkt genau jene Kopien wieder ein, die die Move-Semantik eigentlich vermeiden sollte [3, 9].

Rule of Zero. Die moderne Empfehlung besteht darin, manuelle Ressourcenverwaltung gänzlich zu vermeiden. Speichert eine Klasse ihre Daten in Membern, die ihre Ressourcen bereits selbst korrekt verwalten – etwa `std::vector`, `std::string` oder `std::unique_ptr` –, so sind die vom Compiler erzeugten speziellen Elementfunktionen ihrerseits korrekt, und die Klasse sollte keine der fünf Funktionen deklarieren. Eine Matrix, die einen `std::vector<double>` als Speicher verwendet, folgt der Rule of Zero und erhält korrekte und effiziente Kopier- und Move-Operationen kostenlos. Die in dieser Arbeit untersuchten Implementierungen verwalten den Speicher bewusst manuell, damit das Anlegen, Kopieren, Verschieben und Zerstören des Speicherblocks beobachtbar und messbar wird; in produktivem Code wäre die Rule of Zero der vorzuziehende Entwurf [9, 3].

2.5 Kopier- und Move-Semantik

Die Kopiersemantik beschreibt, was geschieht, wenn ein Objekt dupliziert wird. Der Kopierkonstruktor erzeugt ein neues Objekt als eigenständige Kopie eines bestehenden, während der Kopierzuweisungsoperator den Inhalt eines bereits existierenden Objekts durch eine Kopie eines anderen ersetzt. Für eine ressourcenverwaltende Matrix müssen beide eine tiefe Kopie durchführen: einen neuen Speicherblock gleicher Größe anlegen und jedes Element kopieren. Die Kosten sind daher proportional zur Datenmenge – eine Allokation zuzüglich eines elementweisen Kopiervorgangs der Ordnung $O(n)$ (bzw. $O(n^2)$ für eine $n \times n$ -Matrix). Der Kopierzuweisungsoperator muss zusätzlich den zuvor gehaltenen Speicher freigeben und sich gegen Selbstzuweisung absichern.

Die Move-Semantik beschreibt, was geschieht, wenn die Eigentümerschaft an einer Ressource übertragen statt dupliziert wird. Der Move-Konstruktor und der Move-Zuweisungsoperator nehmen eine rvalue-Referenz (`Matrix&&`) entgegen, die an temporäre Objekte bindet – also an Objekte, die ohnehin gleich zerstört werden und deren Ressourcen daher gefahrlos “gestohlen” werden können. Anstatt zu allokalieren und zu kopieren, übernimmt ein Move-Vorgang lediglich Zeiger und Größe der Quelle und versetzt diese in einen gültigen, leeren Zustand (typischerweise Nullzeiger und Größe null), sodass ihr Destruktor keinen Schaden anrichtet. Die Kosten sind konstant: einige wenige Zeiger- und Ganzzahlzuweisungen, unabhängig von der Größe der Matrix. Es findet weder eine Allokation noch ein elementweises Kopieren statt.

Der Mechanismus beruht auf den Wertkategorien (*value categories*). Ein lvalue besitzt einen Namen und eine dauerhafte Identität; ein rvalue ist ein temporäres Objekt ohne bleibende Identität. Die Move-Überladungen werden automatisch ausgewählt, wenn die Quelle ein rvalue ist – etwa das von `operator+` zurückgegebene temporäre Objekt. Soll ein benanntes Objekt als verschiebbar behandelt werden, kommt `std::move` zum Einsatz; diese Funktion verschiebt selbst nichts, sondern stellt lediglich eine Umwandlung in eine rvalue-Referenz dar, durch die die Move-Überladung ausgewählt wird.

Die praktische Bedeutung für diese Arbeit ergibt sich unmittelbar. In einem Ausdruck wie `C = A + B` erzeugt `operator+` ein temporäres Matrix-Objekt. Steht ausschließlich Kopiersemantik zur Verfügung, so wird dieses temporäre Objekt tief in `C` kopiert – eine Allokation samt elementweisem Kopieren der Ordnung $O(n^2)$ – und anschließend zerstört. Mit Move-Semantik übernimmt `C` stattdessen den Speicherblock des temporären Objekts in konstanter Zeit. Für große Matrizen ist dies der Unterschied zwischen einer Operation, die mit der Datenmenge skaliert, und einer, die es nicht tut – und genau diesen Unterschied quantifizieren die Experimente in Kapitel 4.

2.6 Return Value Optimization (RVO/NRVO)

Die Rückgabe eines Objekts per Wert scheint auf den ersten Blick mit Kosten verbunden zu sein: Die Funktion erzeugt ein lokales Ergebnis und kopiert dieses anschließend zum Aufrufer. Für eine ressourcenverwaltende Matrix bedeutete dies bei jeder Rückgabe eine Allokation sowie einen elementweisen Kopiervorgang der Ordnung $O(n^2)$. In der Praxis beseitigen moderne Compiler den Großteil dieser Kosten durch die Kopierauslassung (*copy elision*) – eine Gruppe von Erlaubnissen, Aufrufe des Kopier- oder Move-Konstruktors selbst dann wegzulassen, wenn diese beobachtbare Seiteneffekte hätten.

Die wichtigste Form ist die Return Value Optimization (RVO). Gibt eine Funktion ein unbenanntes temporäres Objekt – einen *prvalue* – zurück, das unmittelbar in der **return**-Anweisung erzeugt wird, etwa **return Matrix(rows, cols);**, so wird das Objekt direkt im Speicher des Aufrufers konstruiert; es findet weder eine Kopie noch ein Move statt. Seit C++17 ist diese Auslassung verpflichtend: Der Standard betrachtet sie nicht länger als Optimierung, sondern als Sprachgarantie. Sie greift daher auf jeder Optimierungsstufe – einschließlich -O0 – und selbst dann, wenn Kopier- und Move-Konstruktor gelöscht (*deleted*) sind.

Eine zweite Form, die Named Return Value Optimization (NRVO), kommt zum Tragen, wenn eine Funktion eine benannte lokale Variable anstelle eines anonymen temporären Objekts zurückgibt. Der Compiler darf diese Variable direkt im Speicher des Aufrufers konstruieren und so die Kopie bzw. den Move auslassen. Anders als die RVO ist die NRVO jedoch erlaubt, aber nicht garantiert – auch in den aktuellen Standards. Ob sie tatsächlich erfolgt, hängt vom Compiler ab und geschieht in der Regel erst auf höheren Optimierungsstufen.

Wird die NRVO nicht angewandt, greift ein Rückfallmechanismus. Seit C++11 wird der aus einer lokalen Variablen zurückgegebene Wert bei der Überladungsauflösung als rvalue behandelt, sodass der Compiler – sofern vorhanden – den Move-Konstruktor dem Kopierkonstruktor vorzieht. Hierin liegt die entscheidende Verbindung zu den Implementierungsvarianten dieser Arbeit: Bei einer Rückgabe, für die keine Auslassung greift, verursacht die naive Variante (V1, nur Kopie) eine vollständige tiefe Kopie, während die move-fähige Variante (V2) lediglich einen Move in konstanter Zeit benötigt. Die Kopierauslassung beseitigt die Operation vollständig, wenn sie greift; die Move-Semantik begrenzt ihre Kosten, wenn sie nicht greift [6, 3].

Eine wesentliche Einschränkung ist hervorzuheben, da sie die Interpretation der Experimente in Kapitel 4 bestimmt: Die Kopierauslassung gilt für die Initialisierung eines neuen Objekts, nicht für die Zuweisung an ein bereits bestehendes. Die Deklaration **Matrix D = A + B;** initialisiert D unmittelbar aus einem prvalue und wird daher unabhängig von der Variante ausgelassen. Der Ausdruck **C = A + B;** hingegen, bei dem C bereits existiert, lässt sich nicht auslassen – das von **operator+** zurückgegebene temporäre Objekt wird C zugewiesen und ruft dabei in V1 den Kopierzuzuweisungs-, in V2 den Move-Zuweisungsoperator auf. Der Performanceunterschied zwischen den Varianten zeigt sich somit bei der Zuweisung an bestehende Objekte und in verketteten Ausdrücken, nicht bei der direkten Initialisierung.

Über die Auslassung hinaus sind zwei weitere Optimierungen von Bedeutung. Das Inlining, das verstärkt ab -O2 angewandt wird, ersetzt kleine Operatoraufrufe durch ihren Rumpf, beseitigt so den Aufrufaufwand und ermöglicht Optimierungen über die ehemalige Aufrufgrenze hinweg. Bei -O3 wendet der Compiler zusätzlich Schleifenvektorisierung (SIMD) und Loop-Unrolling auf die elementweisen Schleifen an, die die eigentliche arithmetische Arbeit der Ordnung $O(n^2)$ ausmachen. Diese Optimierungen beschleunigen die Berechnung selbst, nicht die Objektverwaltung, weshalb Optimierungsstufe und Implementierungsvariante die Ergebnisse entlang unterschiedlicher Achsen beeinflussen – eine Unterscheidung, die die Benchmarks in Kapitel 4 sichtbar machen.

2.7 Operatorüberladung in anderen Sprachen

Die Art, wie C++ mit der Operatorüberladung umgeht, wird im Vergleich mit Sprachen deutlicher, die zwischen Ausdrucksstärke und Kontrolle merklich unterschiedliche Kompromisse eingehen.

Java verzichtet bewusst auf benutzerdefinierte Operatorüberladung; der einzige eingebaute Fall ist + zur Verkettung von Zeichenketten. Die Sprachgestalter schlossen das Merkmal aus, um den Lesbarkeits- und Wartungsproblemen vorzubeugen, die beliebige Operatordefinitionen mit sich bringen können; eine Matrixaddition muss daher als Methodenaufruf wie `A.add(B)` geschrieben werden. Da alle Objekte Referenztypen sind, die von einem Garbage Collector verwaltet werden, kennt Java weder Wertsemantik noch eine Unterscheidung zwischen Kopie und Move auf Sprachebene; die für diese Arbeit zentralen Effizienzfragen stellen sich in dieser Form nicht, da das Kostenmodell stattdessen von Heap-Allokation und dem Aufwand der Garbage Collection bestimmt wird.

Rust unterstützt die Operatorüberladung über Traits im Modul `std::ops` – etwa `Add`, `Sub`, `Mul` oder `Index`. Der entscheidende Unterschied zu C++ besteht darin, dass Rust standardmäßig verschiebt (*move-by-default*): Die Eigentümerschaft ist Teil des Typsystems und wird vom Borrow Checker zur Übersetzungszeit erzwungen, sodass ein Operand konsumiert (verschoben) wird, sofern der Typ nicht ausdrücklich `Copy` implementiert oder über ein explizites `.clone()` dupliziert wird. Während C++ die Move-Semantik auf ein standardmäßig kopierendes Modell aufsetzte, kehrt Rust diese Voreinstellung um und verlagert die Korrektheitsgarantien, die in C++ von Hand herzustellen sind (die Rule of Five), in den Compiler.

Python bietet die freizügigste Form, und zwar über sogenannte “Dunder”-Methoden wie `__add__`, `__mul__` oder `__getitem__`, einschließlich des eigens für die Matrixmultiplikation eingeführten Operators `@` (`__matmul__`, seit Python 3.5). Die Auflösung erfolgt dynamisch und verursacht pro Operation einen Interpreter-Overhead, weshalb performancekritischer numerischer Code die eigentliche Arbeit an in C implementierte Bibliotheken wie NumPy delegiert, deren Operatoren auf zusammenhängenden Speicherblöcken arbeiten; Ausdrucksstärke und Performance sind damit voneinander entkoppelt.

Für den C++-Teil dieses Sprachvergleichs dienen insbesondere Standardwerk und Referenz als Grundlage [1, 5].

C++ nimmt somit eine besondere Stellung ein: Es legt die vollständigen Kosten der Operatorüberladung – Wert- gegenüber Referenzsemantik, Kopie gegenüber Move, Auslassung – zur Übersetzungszeit und ohne Laufzeit-Overhead in die Hand des Programmierers. Die in dieser Arbeit untersuchten verborgenen Kosten treten gerade deshalb am deutlichsten zutage, weil C++ diese Kontrolle gewährt, statt sie hinter einer Garbage Collection zu verbergen oder durch das Fehlen des Merkmals vorzuenthalten.

3 Praktische Implementierung

Alle in Abschnitt 3 und Abschnitt 4 betrachteten C++-Varianten werden mit dem Sprachstandard `-std=c++17` kompiliert. Diese Festlegung ist wesentlich, da Aussagen zu RVO/Elision und damit zur Sichtbarkeit von Move-Effekten vom Sprachstandard abhängen. Die Messungen verwenden jeweils dieselben Quelltexte und variieren nur die Optimierungsstufe (`-O0`, `-O2`, `-O3`).

Die Implementierung folgt einem didaktischen Stufenmodell. Ausgehend von einer gemeinsamen Datenstruktur werden drei Versionen entwickelt, die sich ausschließlich in den speziellen Elementfunktionen und im Entwurf der Operatoren unterscheiden: Version 1 (nur Kopiersemantik), Version 2 (zusätzlich Move-Semantik) und Version 3 (zusätzlich zusammengesetzte Operatoren). Die für die Laufzeitmessungen verwendete vereinheitlichte `Matrix`-Klasse entspricht funktional Version 3.

3.1 Beschreibung der Beispielklasse

Als Untersuchungsgegenstand dient eine Klasse für dichte Matrizen mit `double`-Elementen. Die Daten werden in einem einzigen, zusammenhängend auf dem Heap allozierten eindimensionalen Feld in Zeilen-Hauptordnung (*row-major*) gehalten; das Element (i, j) liegt am linearen Index $i \cdot \text{cols} + j$. Diese Anordnung ist cache-freundlich, da der sequentielle Zugriff auf benachbarte Elemente benachbarte Cache-Zeilen lädt, und entspricht der in C/C++ sowie in BLAS/LAPACK üblichen Konvention.

```

1  class Matrix {
2  private:
3      double* data;           // zusammenhaengender Heap-Block
4      int rows_, cols_;       // Dimensionen
5      static int instance_count;
6      static int copy_count;   // Zaehler fuer tiefe Kopien
7      static int move_count;   // Zaehler fuer Move-Operationen
8      int index(int row, int col) const { return row * cols_ + col; }
9  public:
10     Matrix(int rows, int cols);           // Konstruktor
11     ~Matrix();                             // Destruktor
12     // ... Rule of Five, Operatoren, Zugriffsmethoden ...
13 };

```

Listing 1: Datenmodell und Instrumentierung der Matrix-Klasse

Neben den arithmetischen Operatoren stellt die Klasse Zugriffsoperatoren (`operator()`, `operator[]`, `at()` jeweils in konstanter und nicht-konstanter Überladung), einen Vergleichsoperator (`operator==` mit Floating-Point-Toleranz) sowie den Stream-Operator `operator<<` bereit. Tabelle 1 fasst den Operatorensatz und die jeweils gewählte Implementierungsform zusammen; die Begründung für die Zuordnung Member / frei folgt der in Abschnitt 2.2 dargestellten Semantik.

Tabelle 1: Operatorenatz der Matrix-Klasse und gewählte Implementierungsform.

Operator	Form	Zweck
Kopier-/Move-Konstruktor	Member	Rule of Five
Kopier-/Move-Zuweisung	Member	Rule of Five
<code>operator()</code> , <code>at()</code>	Member	Elementzugriff
<code>operator[]</code>	Member	Zeilenzugriff
<code>operator+=</code> , <code>-=</code>	Member	In-place-Arithmetik
<code>operator*</code>	Member	Multiplikation $O(n^3)$
<code>operator+</code> , <code>-</code>	frei	symmetrische Arithmetik
<code>operator==</code>	Member	Vergleich
<code>operator<<</code>	frei (friend)	Ausgabe

3.2 Version 1: Naive Implementierung

Version 1 implementiert ausschließlich Kopiersemantik und damit die Rule of Three: Konstruktor, Destruktor, Kopierkonstruktor und Kopierzweisungsoperator. Ein Move-Konstruktor oder eine Move-Zuweisung sind nicht vorhanden; ebenso fehlen zusammengesetzte Operatoren. Der Kopierkonstruktor legt einen neuen Speicherblock an und kopiert die Elemente per `std::memcpy` (tiefe Kopie). Der Kopierzweisungsoperator sichert sich gegen Selbstzuweisung ab und realloziert nur dann, wenn sich die Dimensionen unterscheiden (Listing 2).

```

1  MatrixV1::MatrixV1(const MatrixV1& other)
2      : rows_(other.rows_), cols_(other.cols_) {
3      int size = rows_ * cols_;

```

```

4     data = new double[size];
5     std::memcpy(data, other.data, size * sizeof(double)); // tiefe Kopie
6     copy_count++;
7 }
8
9 MatrixV1& MatrixV1::operator=(const MatrixV1& other) {
10     if (this == &other) return *this;           // Selbstzuweisung
11     if (rows_ != other.rows_ || cols_ != other.cols_) {
12         delete[] data;                          // alten Speicher freigeben
13         rows_ = other.rows_; cols_ = other.cols_;
14         data = new double[rows_ * cols_];        // neu allozieren
15     }
16     std::memcpy(data, other.data, rows_ * cols_ * sizeof(double));
17     copy_count++;
18     return *this;
19 }

```

Listing 2: Tiefe Kopie in Version 1: Kopierkonstruktor und Kopierzweisung

Die arithmetischen Operatoren erzeugen jeweils ein neues Ergebnisobjekt und geben es per Wert zurück. Listing 3 zeigt `operator+`: Das Ergebnis wird lokal konstruiert, elementweise gefüllt und zurückgegeben. Da Version 1 keinen Move-Konstruktor besitzt, fällt die Rückgabe in jenen Fällen, in denen keine Kopierauslassung greift, auf eine vollständige tiefe Kopie zurück.

```

1 MatrixV1 MatrixV1::operator+(const MatrixV1& other) const {
2     MatrixV1 result(rows_, cols_);
3     int size = rows_ * cols_;
4     for (int i = 0; i < size; ++i)
5         result.data[i] = data[i] + other.data[i];
6     return result;                               // Rueckgabe per Wert
7 }

```

Listing 3: Naive Implementierung von `operator+` (V1)

3.3 Version 2: Implementierung mit Move-Semantik

Version 2 vervollständigt die Rule of Five, indem sie Version 1 um einen Move-Konstruktor und einen Move-Zuweisungsoperator ergänzt (Listing 4). Der Move-Konstruktor übernimmt Zeiger und Dimensionen der Quelle und setzt deren Zeiger auf `nullptr`, sodass der Destruktor der ausgehöhlten Quelle keinen Schaden anrichtet; es findet weder eine Allokation noch ein elementweises Kopieren statt. Beide Operationen sind als `noexcept` deklariert – eine Voraussetzung dafür, dass Standardcontainer wie `std::vector` ihre Elemente bei einer Reallokation verschieben statt kopieren.

```

1 MatrixV2::MatrixV2(MatrixV2&& other) noexcept
2     : data(other.data), rows_(other.rows_), cols_(other.cols_) {
3     other.data = nullptr;                       // Quelle aushöhlen
4     other.rows_ = 0; other.cols_ = 0;
5     move_count++;
6 }
7
8 MatrixV2& MatrixV2::operator=(MatrixV2&& other) noexcept {
9     if (this == &other) return *this;
10    delete[] data;                              // eigenen Speicher
11    freigeben

```

```

11     data = other.data;                                // Zeiger uebernehmen
12     rows_ = other.rows_; cols_ = other.cols_;
13     other.data = nullptr; other.rows_ = 0; other.cols_ = 0;
14     move_count++;
15     return *this;
16 }

```

Listing 4: Move-Konstruktor und Move-Zuweisung (V2): Zeigeruebernahme in $O(1)$

Die arithmetischen Operatoren bleiben gegenüber Version 1 unverändert (naive elementweise Berechnung mit Rückgabe per Wert). Der entscheidende Unterschied ist, dass die Rückgabe nun, sofern keine Kopierauslassung greift, einen Move in konstanter Zeit statt einer tiefen Kopie verursacht und dass die Zuweisung aus einem temporären Objekt ($C = A + B$;) den Move-Zuweisungsoperator auswählt.

3.4 Version 3: +=-basierte Implementierung

Version 3 ergänzt die zusammengesetzten Operatoren `operator+=` und `operator-=` als Member-Funktionen, die das linke Objekt in-place verändern und eine Referenz auf `*this` zurückgeben. Die symmetrischen Operatoren `operator+` und `operator-` werden anschließend als freie Funktionen implementiert, die intern auf die zusammengesetzten Operatoren zurückgreifen (Listing 5). Auf diese Weise ist die eigentliche Additionslogik nur an einer Stelle definiert (DRY-Prinzip): Eine Korrektur in `operator+=` wirkt sich automatisch auf `operator+` aus.

```

1  Matrix& Matrix::operator+=(const Matrix& other) {
2      if (rows_ != other.rows_ || cols_ != other.cols_)
3          throw std::invalid_argument("Dimensionen muessen uebereinstimmen");
4      int size = rows_ * cols_;
5      for (int i = 0; i < size; ++i)
6          data[i] += other.data[i];                // in-place, keine Kopie
7      return *this;                                // Referenz fuer Verkettung
8  }
9
10 Matrix operator+(const Matrix& lhs, const Matrix& rhs) {
11     Matrix result = lhs;                          // 1 Kopie des linken
12     // Operanden
13     result += rhs;                                // Member-Operator +=
14     return result;                                // Rueckgabe per Wert
15 }

```

Listing 5: V3: In-place-Operator als Member, freie Funktion auf dessen Basis

Der zusammengesetzte Operator vermeidet in Akkumulationsschleifen (`result += temp`;) jegliche temporären Objekte. Die freie Funktion `operator+` erzeugt hingegen genau eine Kopie des linken Operanden (`Matrix result = lhs`); diese eine Kopie ist der wesentliche Unterschied zwischen dem freien `+` und dem in-place `+=` und schlägt sich, wie Abschnitt 4 zeigt, bei großen Matrizen deutlich in der Laufzeit nieder. Die Multiplikation `operator*` bleibt eine eigenständige Member-Funktion mit dem klassischen dreifach geschachtelten Algorithmus der Ordnung $O(n^3)$.

3.5 Instrumentierung zur Zählung temporärer Objekte

Um die Anzahl der erzeugten Kopien und Moves unabhängig von der Laufzeit zu erfassen, ist die Klasse mit statischen Zählern instrumentiert. Der Kopierkonstruktor und der Kopierzweisungsoperator erhöhen `copy_count`, der Move-Konstruktor und die Move-Zuweisung erhöhen `move_count`. Über die statische Schnittstelle lassen sich die Zähler vor einer Messung zurücksetzen und danach auslesen (Listing 6).

```

1 static void resetStats(); // setzt alle Zaehler auf 0
2 static int getCopyCount(); // Anzahl tiefer Kopien
3 static int getMoveCount(); // Anzahl Move-Operationen
4
5 // Verwendung in einem Benchmark:
6 Matrix::resetStats();
7 for (auto _ : state) { Matrix result = A + B; benchmark::DoNotOptimize(result
8 ); }
9 // Anschliessend: Matrix::getCopyCount(), Matrix::getMoveCount()

```

Listing 6: Statische Zaehler zur Messung von Kopien und Moves

Wichtig für die Interpretation ist, dass diese Zähler *Operationen* und nicht allozierte Bytes messen: Ein erhöhter `copy_count` entspricht einer tiefen Kopie (Allokation plus elementweises Kopieren), ein erhöhter `move_count` einer reinen Zeigerübernahme. Diese Instrumentierung bildet die Grundlage der in Abschnitt 4.4 und Abschnitt 4.5 ausgewerteten Kennzahlen.

4 Experimente und Analyse

4.1 Versuchsaufbau

Da seit C++17 die Materialisierung eines prvalue-Rückgabewerts verpflichtend ohne Kopie oder Move erfolgt, unterscheiden sich naive und move-fähige Implementierungen im Fall der direkten Initialisierung (`Matrix D = A + B;`) in der Regel kaum. Reale Unterschiede sind dagegen bei der Zuweisung aus einem temporären Objekt, bei verketteten Ausdrücken mit echten Zwischenwerten sowie bei Container-Reallokationen zu erwarten. Der Versuchsaufbau ist entsprechend so gewählt, dass die unterscheidenden Primitive der drei Versionen einzeln gemessen werden:

1. **Operationskosten:** Addition, Subtraktion, Multiplikation und verkettete Ausdrücke (Initialisierung aus prvalue).
2. **Kopie vs. Move:** Kopier- gegen Move-Konstruktor sowie Kopier- gegen Move-Zuweisung (isoliert den Beitrag von Version 2 gegenüber Version 1).
3. **Freie Funktion vs. in-place:** `A + B` gegen `A += B` über mehrere Matrixgrößen (isoliert den Beitrag von Version 3 gegenüber Version 2).
4. **Skalierung:** Addition und Multiplikation über einen Größenbereich.
5. **Container und Akkumulation:** Einfügen in `std::vector<Matrix>` sowie Akkumulationsschleifen.
6. **Temporäre Objekte:** Zählung der Kopien und Moves je Operation.

Sämtliche Laufzeitmessungen wurden auf der in Tabelle 2 beschriebenen Umgebung durchgeführt. Für jede Konfiguration werden identische Eingangsdaten und Compiler-Flags verwendet, sodass beobachtete Unterschiede aus den Implementierungsvarianten und nicht aus veränderten Randbedingungen stammen.

4.2 Messmethodik und Reproduzierbarkeit

Die Laufzeitmessung erfolgt mit Google Benchmark. Das Framework bestimmt die Iterationszahl automatisch anhand einer Mindestlaufzeit (`--benchmark_min_time=0.05s`), führt mehrere Messläufe mit wachsender Iterationszahl durch und aggregiert die Ergebnisse statistisch. Der Aufruf

Tabelle 2: Mess- und Systemumgebung.

Komponente	Spezifikation
Prozessor	16 Kerne, 3800 MHz
Cache	L1 32 KiB ($\times 8$), L2 512 KiB ($\times 8$), L3 32 MiB
Compiler	Clang 22.1.6 (llvm-mingw)
Sprachstandard	C++17
Messframework	Google Benchmark 1.9.5
Optimierung	-O0, -O2, -O3

`benchmark::DoNotOptimize` verhindert, dass der Compiler die zu messende Berechnung als toten Code entfernt. Im erweiterten Benchmark werden die Matrizen mit festem Zufallssamen ($SEED = 42$) initialisiert, um Reproduzierbarkeit sicherzustellen. Die Kopier- und Move-Zähler werden über die in Abschnitt 3.5 beschriebene Instrumentierung erfasst.

Die in dieser Arbeit verwendete Profilierung des Speicherverhaltens stützt sich auf diese Zähler und nicht auf externe Werkzeuge: `valgrind` `massif` und `perf` standen auf der eingesetzten Windows-/llvm-mingw-Plattform nicht zur Verfügung. Die in Abschnitt 4.5 berichteten Kennzahlen sind daher als Anzahl von Kopier- und Move-Operationen zu verstehen.

Grenzen der Messung. Mehrere Mikrobenchmarks (Move-Konstruktor, Move-Zuweisung sowie die in-place-Varianten bei kleinen Matrizen) kapseln ihre Vorbereitung in `state.PauseTiming()/ResumeTiming()`. Das Pausieren und Fortsetzen der Zeitmessung verursacht selbst einen Aufwand in der Größenordnung von Mikrosekunden und dominiert daher Operationen, die – wie ein Move – nur wenige Nanosekunden benötigen. Die entsprechenden Vergleiche (Abschnitt 4.6) sind deshalb nicht als belastbare Messung der reinen Move-Kosten zu werten. Belastbar sind hingegen jene Messungen, die ohne Pausieren auskommen: die Operations- und Skalierungsmessungen sowie der Vergleich von freier Funktion und in-place-Operator bei größeren Matrizen. Weitere Einschränkungen sind die Beschränkung auf eine einzige Maschine sowie ein unvollständiger erweiterter -O3-Lauf.

4.3 Ergebnisse: Laufzeit

Tabelle 3 zeigt die Laufzeit der Kernoperationen über die drei Optimierungsstufen. Der Übergang von -O0 zu -O2 bringt den mit Abstand größten Gewinn: Die Addition einer 100×100 -Matrix beschleunigt sich von rund $18,8 \mu s$ auf $3,1 \mu s$ (Faktor 6,2), die Multiplikation von rund $796 \mu s$ auf $61 \mu s$ (Faktor 13). Der weitere Schritt zu -O3 bringt dagegen keinen konsistenten Vorteil; in mehreren Fällen ist -O3 sogar geringfügig langsamer (Abbildung 1). Für die speicherbandbreitenbegrenzte elementweise Addition liefert die zusätzliche Vektorisierung bei -O3 kaum Nutzen, während die Multiplikation bereits bei -O2 weitgehend ausoptimiert ist.

Tabelle 3: Laufzeit der Kernoperationen je Optimierungsstufe (in ns).

Operation	-O0	-O2	-O3
Addition 100×100	18 835	3 052	3 864
Subtraktion 100×100	21 625	4 039	3 878
Multiplikation 50×50	795 848	60 827	61 417
In-place += 100×100	20 170	3 177	3 897
Verkettet $(A+B)-(A-B)$	58 763	11 179	12 130

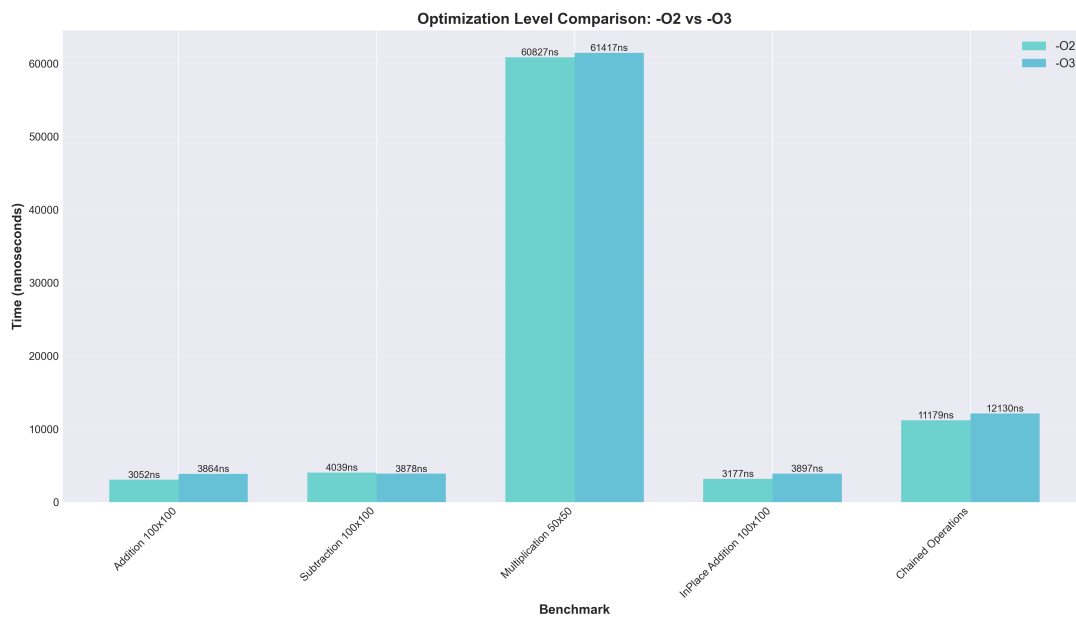


Abbildung 1: Laufzeitvergleich -O2 gegenüber -O3 für ausgewählte Operationen. Die höchste Optimierungsstufe bringt gegenüber -O2 keinen durchgängigen Vorteil.

Die Multiplikation dominiert das Laufzeitprofil deutlich: In einer gemischten Arbeitslast aus Addition, Subtraktion, Multiplikation, in-place-Operation und verketteten Ausdrücken entfällt der weitaus größte Anteil auf die Multiplikation (Abbildung 2). Dies entspricht der Komplexität von $O(n^3)$ gegenüber $O(n^2)$ der übrigen Operationen.

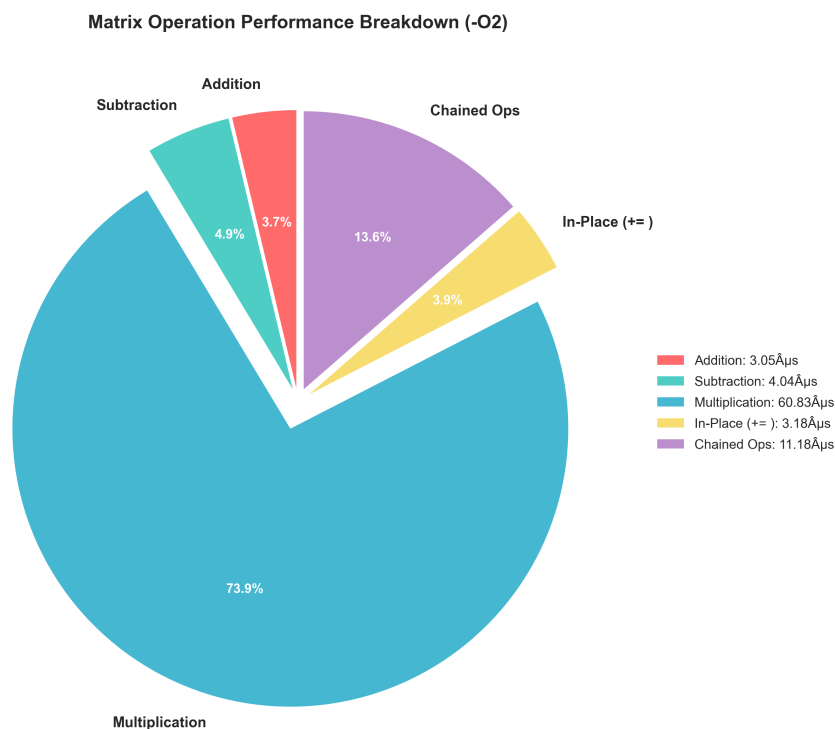


Abbildung 2: Anteil der Operationen an der Gesamtlaufzeit (-O2). Die Multiplikation überwiegt aufgrund ihrer kubischen Komplexität.

Die Skalierung der Addition mit der Matrixgröße zeigt das erwartete $O(n^2)$ -Verhalten, allerdings überlagert von Cache-Effekten (Tabelle 4, Abbildung 3). Beim Übergang von 100×100 auf 500×500 wächst die Datenmenge um den Faktor 25, die Laufzeit jedoch um etwa den Faktor 199. Eine

500×500 -Matrix belegt rund 2 MB und überschreitet damit den L2-Cache deutlich; die Operation wird speicherbandbreitenbegrenzt. Der erweiterte Benchmark bestätigt diese Tendenz bis 1000×1000 (rund 2,17 ms bei -O2).

Tabelle 4: Laufzeit der Addition in Abhängigkeit von der Matrixgröße (-O2).

Größe	Zeit	Faktor (ggü. 10×10)
10×10	82 ns	1
50×50	1 135 ns	13,8
100×100	3 052 ns	37,2
500×500	607 987 ns	7 414

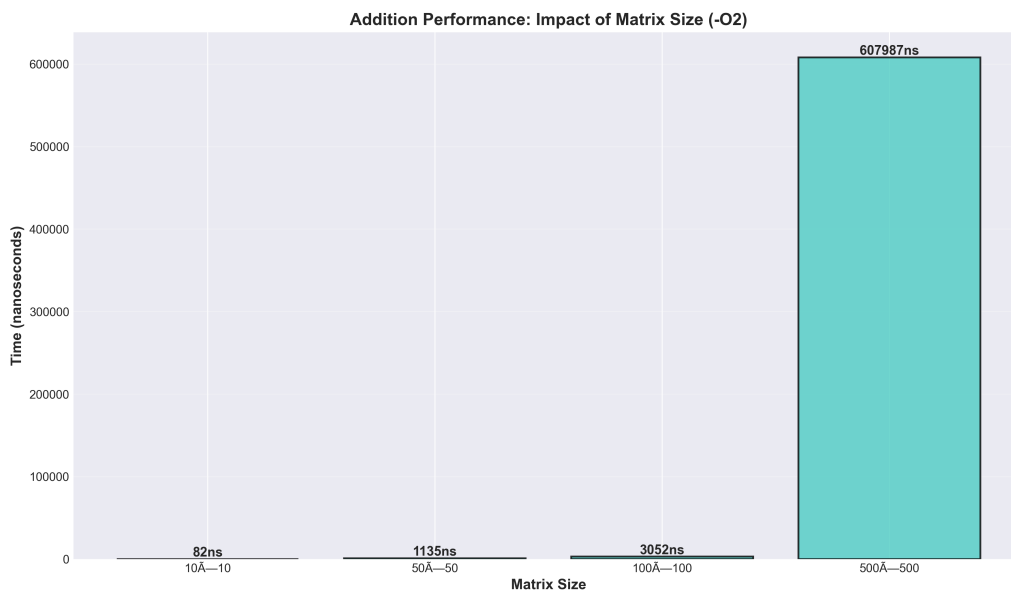


Abbildung 3: Laufzeit der Addition über die Matrixgröße (-O2). Der überproportionale Anstieg bei 500×500 ist auf Cache-Effekte zurückzuführen.

Den belastbarsten Hinweis auf den Nutzen des in-place-Entwurfs liefert der direkte Vergleich von freier Funktion ($A + B$) und in-place-Operator ($A += B$). Bei 100×100 ist die in-place-Variante etwa 25 % schneller ($3,46 \mu s$ gegenüber $4,61 \mu s$), bei 500×500 rund das 2,6-Fache ($235 \mu s$ gegenüber $609 \mu s$). Der Unterschied entspricht gerade jener einen Kopie des linken Operanden, die `operator+` intern anlegt (Listing 5) und die `operator+=` einspart. Bei sehr kleinen Matrizen (10×10) kehrt sich das Bild messtechnisch um; dieser Effekt ist jedoch auf den in Abschnitt 4.2 beschriebenen Pausierungs-Aufwand zurückzuführen und nicht auf die Operation selbst.

4.4 Ergebnisse: Temporäre Objekte

Tabelle 5 weist die je Operation erzeugten Kopien und Moves aus. Bemerkenswert ist, dass über alle Operationen hinweg kein einziger Move gezählt wird. Dies ist kein Widerspruch zur vorhandenen Move-Semantik, sondern eine unmittelbare Folge der Kopierauslassung: Die freie Funktion `operator+` gibt eine benannte lokale Variable zurück (NRVO), und am Aufrufort initialisiert ein `prvalue` das Ergebnisobjekt (garantierte Elision). Beide Mechanismen entfernen den Rückgabe-Move vollständig. Die gezählten Kopien stammen daher nicht aus der Rückgabe, sondern aus der einen Kopie des linken Operanden innerhalb von `operator+` bzw. aus der bewussten Kopie zur Vorbereitung der in-place-Messung.

Damit bestätigt sich die theoretische Vorhersage aus Abschnitt 2.6: Verkettete Ausdrücke erzeugen proportional mehr Kopien (zwei statt einer), während die Multiplikation – die ihr Ergebnis intern aufbaut, ohne den linken Operanden zu kopieren – ganz ohne Kopie auskommt. Abbildung 4 stellt die absoluten

Tabelle 5: Kopien und Moves je Operation (instrumentierte Zählung).

Operation	Kopien / Op.	Moves / Op.
Addition ($A + B$)	1	0
Subtraktion ($A - B$)	1	0
Multiplikation ($A \cdot B$)	0	0
In-place ($temp = A$; $temp += B$)	1	0
Verkettet ($A + B + C$)	2	0
Gemischt	2	0

Zählwerte dar; da Google Benchmark je Operation unterschiedlich viele Iterationen ausführt, sind für den Vergleich die Werte *pro Operation* aus Tabelle 5 maßgeblich.

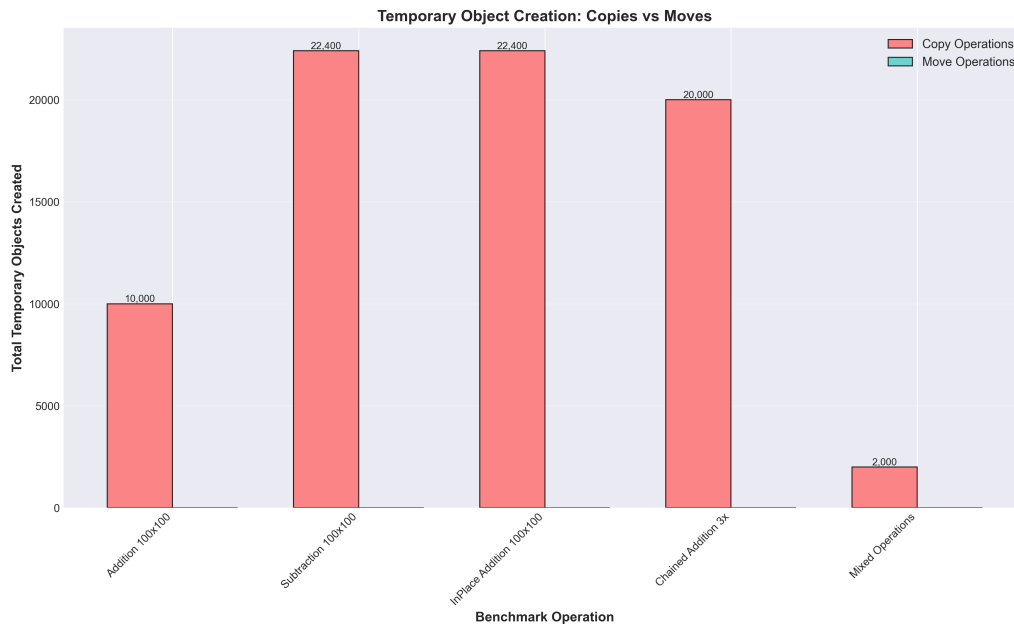


Abbildung 4: Erzeugte temporäre Objekte (Kopien gegenüber Moves). Die Move-Operationen entfallen aufgrund der Kopierauslassung vollständig.

4.5 Ergebnisse: Speicherverbrauch

Das Speicherverhalten wird anhand der Kopier- und Move-Zähler in fünf Szenarien untersucht (Tabelle 6). Da die vereinheitlichte Klasse Move-Semantik besitzt, gibt die Spalte “gemessen” die tatsächlich gezählten Operationen wieder; die Spalte “V1-Schätzung” gibt analytisch an, wie viele tiefe Kopien dieselben Szenarien ohne Move-Semantik verursachen würden (jede Move-Operation wird dann zu einer Kopie).

Tabelle 6: Kopier- und Move-Operationen je Szenario (instrumentierte Zählung; V1-Spalte als analytische Schätzung).

Szenario	Kopien	Moves	V1-Schätzung (Kopien)
Einfache Addition ($m_3 = m_1 + m_2$)	1	1	2
Verkettete Addition ($m_4 = m_1 + m_2 + m_3$)	2	1	3
Akkumulation ($+=$, $100\times$)	0	0	100
Vektor (100 Matrizen)	100	127	> 100
Temporäre Ausdrücke ($1000\times$)	2 000	1 000	3 000

Die Akkumulationsschleife mit `operator+=` kommt ohne jede Kopie und ohne jeden Move aus – die Datenmengen werden ausschließlich in-place addiert. Ohne zusammengesetzten Operator (`result = result + temp;`) wären je Iteration eine Kopie und eine Zuweisung nötig. Beim Einfügen in einen `std::vector` treten neben den 100 Kopien der Elemente zusätzliche Move-Operationen bei den Re-

allokationen des Vektors auf; diese sind nur möglich, weil der Move-Konstruktor `noexcept` ist (Abschnitt 2.4). Abbildung 5 stellt die gemessenen Operationen der vereinheitlichten Klasse der analytischen V1-Schätzung gegenüber.

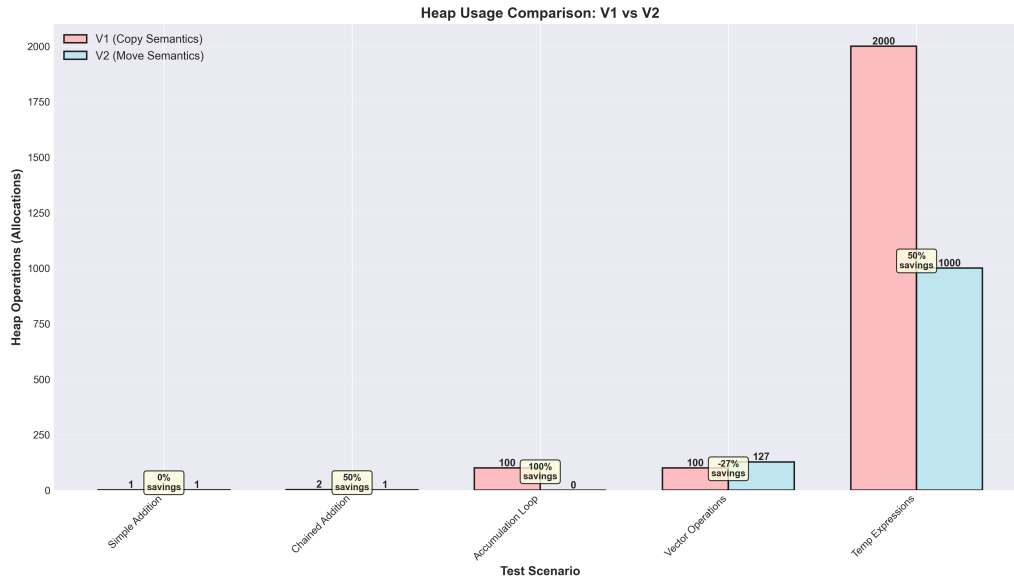


Abbildung 5: Kopier- und Move-Operationen je Szenario (V1 als analytische Schätzung gegenüber der move-fähigen Implementierung V2).

4.6 Interpretation der Ergebnisse

Die Ergebnisse lassen sich zu vier belastbaren Aussagen verdichten.

Erstens dominiert die algorithmische Komplexität die Laufzeit weit stärker als der Entwurf der Operatoren. Die Multiplikation ($O(n^3)$) übersteigt Addition und Subtraktion ($O(n^2)$) um mehr als eine Größenordnung und bestimmt das Laufzeitprofil gemischter Arbeitslasten (Abbildung 2).

Zweitens liegt der mit Abstand größte Optimierungsgewinn im Übergang von `-00` zu `-02`, während `-03` keinen verlässlichen Zusatznutzen bringt. Die Optimierungsstufe wirkt auf die Berechnungsschleifen, nicht auf die Objektverwaltung; beide Achsen sind weitgehend unabhängig.

Drittens zeigt sich der praktische Nutzen des in-place-Entwurfs am deutlichsten dort, wo die Messung nicht durch den Pausierungs-Aufwand verfälscht wird: Bei 500×500 ist `+=` rund 2,6-mal schneller als das freie `+`, da letzteres eine vollständige Kopie des linken Operanden anlegt. Dies ist der unmittelbar messbare Vorteil von Version 3 gegenüber Version 2.

Viertens bestätigt die Zählung der temporären Objekte die theoretische Erwartung: Unter C++17 entfernt die Kopierauslassung sämtliche Rückgabe-Moves, weshalb sich naive und move-fähige Implementierungen bei der direkten Initialisierung (`Matrix D = A + B;`) kaum unterscheiden. Der Nutzen der Move-Semantik verlagert sich damit auf die Zuweisung aus temporären Objekten, auf verkettete Ausdrücke und insbesondere auf Container-Reallokationen, wie Tabelle 6 zeigt.

Einschränkend ist festzuhalten, dass die direkten Mikrobenchmarks von Kopier- gegen Move-Konstruktor (gemessen 1523 ns gegenüber 1175 ns) und von Kopier- gegen Move-Zuweisung (1382 ns gegenüber 1552 ns, Abbildung 6) den theoretisch erwarteten großen Vorsprung der Move-Operationen *nicht* widerspiegeln. Ursache ist der in Abschnitt 4.2 beschriebene Pausierungs-Aufwand, der die wenige Nanosekunden dauernde Zeigerübernahme überdeckt. Der eigentliche Beleg für den Nutzen der Move-Semantik liegt daher nicht in diesen Einzelmessungen, sondern in der Kombination aus den Operationszählungen (Tabelle 5, 6) und dem skalierungsabhängigen Vorteil des in-place-Entwurfs.

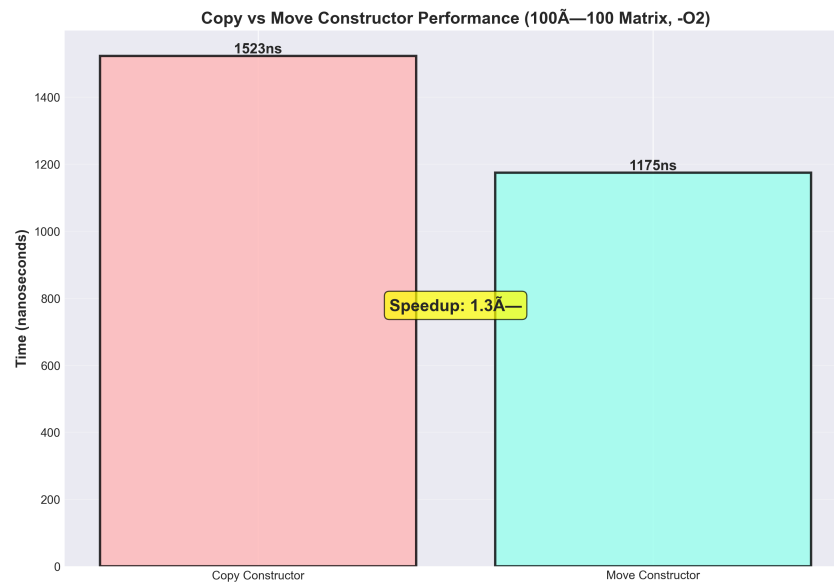


Abbildung 6: Kopier- gegenüber Move-Konstruktor (100×100 , -O2). Der geringe gemessene Abstand ist durch den Pausierungs-Aufwand des Messrahmens bedingt und unterschätzt den tatsächlichen Vorteil der Move-Operation.

5 Fazit

5.1 Zusammenfassung

Die vorliegende Arbeit hat das Überladen von Operatoren in C++ zunächst theoretisch dargestellt und seine Effizienzaspekte anschließend am Beispiel einer Matrix-Klasse empirisch untersucht. Ausgangspunkt war die Beobachtung, dass die syntaktische Eleganz überladener Operatoren erhebliche, im Quelltext nicht unmittelbar sichtbare Kosten verbergen kann – die Erzeugung temporärer Objekte, tiefe Kopien und wiederholte Heap-Allokationen. Die Untersuchung ging der Frage nach, unter welchen Bedingungen unterschiedliche Implementierungsvarianten (naive Kopiersemantik, Move-Semantik und zusammengesetzte Operatoren) diese Kosten tatsächlich beeinflussen.

Die Ergebnisse lassen sich in vier Aussagen zusammenfassen. *Erstens* dominiert die algorithmische Komplexität die Laufzeit weit stärker als der Entwurf der Operatoren: Die Multiplikation mit $O(n^3)$ übersteigt die elementweisen Operationen mit $O(n^2)$ um mehr als eine Größenordnung. *Zweitens* liegt der größte Optimierungsgewinn im Übergang von -O0 zu -O2 (Faktor 6 bis 13), während -O3 keinen verlässlichen Zusatznutzen bringt; die Optimierung wirkt auf die Berechnungsschleifen, nicht auf die Objektverwaltung. *Drittens* entfernt die seit C++17 verpflichtende Kopierauslassung sämtliche Rückgabemoves, weshalb sich naive und move-fähige Implementierungen bei der direkten Initialisierung (`Matrix D = A + B;`) praktisch nicht unterscheiden – in den Messungen wurde kein einziger Move gezählt. Der Nutzen der Move-Semantik verlagert sich damit auf die Zuweisung aus temporären Objekten, auf verkettete Ausdrücke und vor allem auf Container-Reallokationen. *Viertens* zeigt sich der praktische Vorteil des in-place-Entwurfs am deutlichsten beim Vergleich von freier Funktion und zusammengesetztem Operator: Bei einer 500×500 -Matrix ist `+=` rund 2,6-mal schneller als das freie `+`, da Letzteres eine vollständige Kopie des linken Operanden anlegt, die `+=` einspart.

Damit bestätigt sich die Ausgangsthese in präzisierter Form: Lesbarer und performanter Code sind nicht zwangsläufig identisch, lassen sich aber durch geeignete Entwurfstechniken miteinander vereinbaren. Move-Semantik, zusammengesetzte Operatoren und das Vertrauen auf die compilerseitige Kopierauslassung erlauben es, die ausdrucksstarke Operatorsyntax beizubehalten, ohne ihre verborgenen Kosten in Kauf nehmen zu müssen. Einschränkend ist festzuhalten, dass einige Mikrobenchmarks – insbesondere

der direkte Vergleich von Kopier- und Move-Operationen – durch den Aufwand des Messrahmens überlagert werden und den theoretischen Vorteil der Move-Semantik unterschätzen; die belastbaren Belege liefern stattdessen die Operationszählungen und der skalierungsabhängige Vergleich von + und +=.

5.2 Empfehlungen

Aus den theoretischen und empirischen Erkenntnissen lassen sich die folgenden praktischen Empfehlungen für den Einsatz der Operatorüberladung ableiten:

1. **Rule of Five vollständig umsetzen.** Verwaltet eine Klasse eine eigene Ressource, sollten alle fünf speziellen Elementfunktionen deklariert werden; Move-Konstruktor und Move-Zuweisung sind als `noexcept` zu kennzeichnen, damit Standardcontainer ihre Elemente verschieben statt kopieren. Wo möglich, ist die Rule of Zero (Speicherung in `std::vector` o. Ä.) vorzuziehen, da sie korrekte Kopier- und Move-Operationen kostenlos liefert [3, 9].
2. **Zusammengesetzte Operatoren als Member, symmetrische als freie Funktionen.** Die Kernlogik gehört in `operator+=`; `operator+` wird als freie Funktion darauf aufgebaut. Das vermeidet Codeduplizierung (DRY) und behandelt beide Operanden gleichberechtigt. In Akkumulationsschleifen ist += dem wiederholten + vorzuziehen, da es jegliche Kopien vermeidet [8, 2].
3. **Rückgabe per Wert nicht vorschnell meiden.** Unter C++17 beseitigt die Kopierauslassung die Kosten der Rückgabe per Wert bei direkter Initialisierung vollständig. Optimierungsaufwand ist gezielt dort zu investieren, wo Move-Semantik wirklich greift: bei der Zuweisung aus temporären Objekten, bei verketteten Ausdrücken und bei Container-Reallokationen.
4. **Optimierungsstufe messen statt annehmen.** -O2 stellt einen guten Standard dar; -O3 ist nicht automatisch schneller und sollte fallweise gemessen werden.
5. **Zuerst Algorithmus und Datenlayout, dann Objektverwaltung.** Da die algorithmische Komplexität die Laufzeit dominiert, sind Wahl des Algorithmus und ein cache-freundliches Speicherlayout vorrangig; die Feinheiten der Kopier- und Move-Semantik entfalten ihre Wirkung erst danach.
6. **Erwartungskonformität wahren.** Überladene Operatoren sollten sich semantisch wie ihre eingebauten Vorbilder verhalten; insbesondere sind Überladungen von `&&` und `||` wegen des Verlusts der Kurzschlussauswertung zu vermeiden [5, 9].

5.3 Ausblick

Mehrere Fragestellungen blieben außerhalb des Rahmens dieser Arbeit und bieten Ansatzpunkte für weiterführende Untersuchungen. Naheliegend ist die Implementierung von *Expression Templates*, die verkettete Ausdrücke wie `A + B + C + D` ohne Zwischenobjekte in einem einzigen Durchlauf auswerten und damit die in dieser Arbeit gezählten Kopien temporärer Operanden vollständig eliminieren könnten. Ihr Nutzen wäre gegen die deutlich erhöhte Implementierungskomplexität und die längeren Übersetzungszeiten abzuwägen [7, 4].

Auf methodischer Ebene wäre eine verfeinerte Messung wünschenswert, die die nur wenige Nanosekunden dauernden Move-Operationen erfasst, ohne durch das Pausieren und Fortsetzen der Zeitmessung verfälscht zu werden – etwa durch die Messung großer Operationsbündel oder durch Inspektion des erzeugten Maschinencodes. Ebenso könnte eine Profilierung des tatsächlichen Speicherverbrauchs mit plattform-spezifischen Werkzeugen (`valgrind massif`, `perf`) unter Linux die hier zählerbasierte Analyse durch Messungen des allozierten Speichers in Bytes ergänzen.

Inhaltlich ließe sich die Untersuchung auf cache-bewusste, blockweise Multiplikationsverfahren, explizite Vektorisierung (SIMD) und die Parallelisierung großer Matrizen ausweiten. Schließlich böte ein Vergleich der manuell speicherverwaltenden Implementierung mit einer Rule-of-Zero-Variante sowie der Einsatz des Drei-Wege-Vergleichs `operator<=>` aus C++20 weitere Anknüpfungspunkte, um Effizienz und modernen Sprachstand miteinander zu verbinden.

Quellenverzeichnis

- [1] Stroustrup, Bjarne: *The C++ Programming Language*. 4. Auflage, Addison-Wesley, 2013.
- [2] Meyers, Scott: *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*. 3. Auflage, Addison-Wesley, 2005.
- [3] Meyers, Scott: *Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14*. O'Reilly Media, 2014.
- [4] Vandevorde, David; Josuttis, Nicolai M.; Gregor, Douglas: *C++ Templates: The Complete Guide*. 2. Auflage, Addison-Wesley, 2017.
- [5] cppreference.com: *operator overloading*. <https://en.cppreference.com/w/cpp/language/operators> (abgerufen am 28.05.2026).
- [6] cppreference.com: *copy elision*. https://en.cppreference.com/w/cpp/language/copy_elision (abgerufen am 29.05.2026).
- [7] Veldhuizen, Todd: *Expression Templates*. C++ Report, Vol. 7, No. 5, 1995.
- [8] Sutter, Herb; Alexandrescu, Andrei: *C++ Coding Standards: 101 Rules, Guidelines, and Best Practices*. Addison-Wesley, 2004.
- [9] ISO C++ Foundation: *C++ Core Guidelines*. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines> (abgerufen am 28.05.2026).